Package 'vbayesGP'

November 8, 2026

Type Package

Title Gaussian Variational Approximation to Gaussian Process Regression

Version 1.5.0

Date 2026-11-08

Author Seongil Jo [aut, cre],
Woojoo Lee [aut]

Maintainer Seongil Jo <bstatsjo@gmail.com>

Description Implements Gaussian variational approximation to Bayesian semiparametric regression with a Gaussian process prior built on the radial basis function (RBF) kernel. Continuous outcomes are fitted directly, and binary and count outcomes by Polya-Gamma augmentation with a sparse inducing-point approximation to the process. Exposures may enter through a single kernel or through several weighted indices, giving a multiple index model; an index model with a Polya-Gamma outcome takes binary and count outcomes, its bound integrating the inducing values out analytically. The lengthscale parameters take a normal, an independent normal or a continuous shrinkage prior; under shrinkage the fit reports posterior inclusion probabilities for variable selection. Cluster-level random intercepts and slopes are available for every model, and mini-batch stochastic optimisation for the fixed-effects fits, the whole sample being used whenever random effects are present.

License GPL-2

Imports Rcpp (>= 1.0.8), fields, ggplot2, ks, dplyr, magrittr, tibble, tidyr

LinkingTo Rcpp, RcppArmadillo, BH

Suggests bkmr, MASS, knitr, rmarkdown

VignetteBuilder knitr

Depends R (>= 3.5.0)

RoxygenNote 7.3.3

Encoding UTF-8

LazyData true

LazyLoad yes

NeedsCompilation yes

Contents

computeOverallRisk . . . 2
computePPIPs . . . 3
computeSingVarInt . . . 5
computeSingVarRisk . . . 6
extractELBO . . . 7
extractPostSamps . . . 8
extractRandomEffects . . . 9
fitted.gpmim . . . 9
fitted.gpr . . . 10
gvaggpmim . . . 12
gvaggpr . . . 16
gvagpmim . . . 21
gvagpr . . . 25
metals . . . 29
negbinomial . . . 30
plot.gpmim . . . 31
plot.gpr . . . 32
plot.gprfitBivar . . . 33
plot.gprfitBivarLevels . . . 33
plot.gprfitUnivar . . . 34
plot.risks . . . 34
predict.gpmim . . . 35
predict.gpr . . . 36
predictorResponseBivar . . . 37
predictorResponseBivarLevels . . . 38
predictorResponseBivarPair . . . 39
predictorResponseUnivar . . . 41
print.gpmim . . . 42
print.gpr . . . 43
summary.gpmim . . . 43
summary.gpr . . . 44

Index **46**

`computeOverallRisk` *Calculate Overall Risk Summaries*

Description

Compare estimated `f` function when all predictors are at a particular quantile to when all are at a second fixed quantile

Usage

```
computeOverallRisk(
  object,
  qs = seq(0.25, 0.75, by = 0.05),
  q.fixed = 0.5,
  nsamples = 1000,
  ...
)
```

Arguments

object	an object of class gpr obtained from **gvagpr** function
qs	vector of quantiles at which to calculate the overall risk summary
q.fixed	a second quantile at which to compare the estimated f function
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
...	unused argument

Value

a data frame containing the (posterior mean) estimate and posterior standard deviation of the overall risk measures

References

Bobb J (2023). _bkmr: Bayesian Kernel Machine Regression_. R package version 0.2.2, <https://github.com/jenfb/bkmr>

See Also

gvagpr

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
risks.overall <- computeOverallRisk(fout, qs = seq(0.25, 0.75, by = 0.05),
                                    q.fixed = 0.5, nsamples = 300)
risks.overall$summary
plot(risks.overall)
```

---

computePPIPs *Compute pseudo posterior inclusion probabilities (PPIPs) from VGPR model or VGPMIM fits*

Description

Compute pseudo posterior inclusion probabilities (PPIPs) using Sequential-2-Means from Variational Gaussian Process Regression (VGPR) model fit or Variational Gaussian Process Multiple Index Model (VGPMIM) fit

Usage

```
computePPIPs(object, nsamples = 1000, ntuning = 10)
```

Arguments

object	an object of class gpr or gpmim
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
ntuning	the number of chosen values of the tuning parameter for variable selection

Value

a data frame of pseudo posterior inclusion probabilities. For a gpr fit there is one row per exposure. For a gpr fit with a grouped prior (group given to gvagpr or gvaggpr) there is in addition one row per group, placed first. A group counts as selected in a posterior draw when at least one of its members is in the signal cluster of the exposures, the group-level inclusion of the grouped selection in **bkmr**, or when its total inverse length-scale T_g is in the signal cluster of the same sequential two-means step applied to the G totals; the second condition admits a group whose members are each too small to be separated from the noise but whose total is not. The group's PPIP is the proportion of such draws, so it is never below any member's probability. The group rows are read only when the exposure level selects something: if no exposure has a PPIP above one half, two-means has split noise (see below), the group probabilities would only collect that forced selection, about L_g/M for a group of L_g members, and they are reported as zero instead. The exposure rows then carry the group they belong to (group), that group's probability (group.PPIP) and the posterior mean share $w_m = \gamma_m/T_g$ of the group's total inverse length-scale $T_g = \sum_{m \in g} \gamma_m$ (cond.PPIP), which sums to one within a group and says which members carry a selected group, as the conditional inclusion probability of **bkmr** does; the totals themselves, with credible intervals, are the T. rows of summary. For a gpmim fit there are two levels: one row per index, from the index length-scales γ_m, and one row per exposure, from the unnormalised index weights $\psi_{ml} = \sqrt{\gamma_m}\,\alpha_{ml}$. The two nest, since $\gamma_m = \sum_l \psi_{ml}^2$. The normalised weights α_{ml} are not used: they lie on the unit sphere and so cannot be shrunk toward zero, which is what the sequential two-means post-processing assumes.

Rows carry the names the fit was given. Index rows take the element names of a named list Z, exposure rows the column names of the groups, and anything left unnamed falls back to Z1, Z2, ... and Z1_1, Z1_2, ..., where the trailing number counts within the index rather than across the whole of Z.

A gpmim fit adds two further columns, index and index.PPIP, which name the index an exposure belongs to and repeat that index's own probability. They are there because the two levels are read together: a weight row says which exposures carry an index only when that index is itself selected. Where an index is excluded, its weight rows are not a statement about which of its exposures survive; sequential two-means always returns two non-empty clusters, so with no signal to separate it splits noise, and some of those weights come back above one half. If no index is selected at all, that is true of every weight row. A weight above one half sitting under an index.PPIP below one half is that situation and not a selected exposure. Both levels are reported in full and neither is blanked out, since the weight level is the more accurate of the two and gating it on the index level would propagate the index level's misses.

Sequential two-means separates the larger values from the smaller ones, so it answers a comparative question and not an absolute one. Two-means returns two non-empty clusters, so of M values it selects at least one and at most $M-1$ in every posterior draw, and the probabilities sum to between 1 and $M-1$. The procedure can therefore report that some are active but never that all of them are, nor that none of them are. With nothing active the forced selection is spread over the candidates, so no single probability need reach one half once there are three or more of them; with everything active at least one genuine signal is called null. The bounds meet at $M = 2$, where exactly one of the two is selected in every draw and the two probabilities sum to one: for a gpmim fit with two indices the index rows say which index is the larger, not whether either matters, and a warning is given.

Whether an index or an exposure matters at all is read off `summary` instead, where the quantities are compared with zero rather than with each other: `lambda` is the amplitude of the exposure-response surface, `gamma.` gives each index its own length-scale, and the signed `psi.` rows give each exposure inside its index, all with credible intervals.

Author(s)

Seongil Jo

References

Li, H. and Pati, D. (2017). "Variable Selection Using Shrinkage Priors", Computational Statistics and Data Analysis, 107, 107-119.

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

`computeSingVarInt` *Single Variable Interaction Summaries*

Description

Compare the single-predictor health risks when all of the other predictors in Z are fixed to their a specific quantile to when all of the other predictors in Z are fixed to their a second specific quantile.

Usage

```
computeSingVarInt(
  object,
  which.z,
  qs.diff = c(0.25, 0.75),
  qs.fixed = c(0.25, 0.75),
  nsamples = 1000,
  ...
)
```

Arguments

`object`	an object of class `gpr` obtained from **gvagpr** function
`which.z`	vector indicating which variables (columns of `Z`) for which the summary should be computed
`qs.diff`	vector indicating the two quantiles at which to compute the single-predictor risk summary
`qs.fixed`	vector indicating the two quantiles at which to fix all of the remaining exposures in `Z`
`nsamples`	(positive integer), number of posterior samples to draw and save, defaults to 1000
`...`	unused argument

Value

a data frame containing the (posterior mean) estimate and posterior standard deviation of the single-predictor risk measures

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
risks.int <- computeSingVarInt(fout, which.z = 1:3, nsamples = 300)
risks.int$summary
plot(risks.int)
```

---

computeSingVarRisk *Single Variable Risk Summaries*

Description

Compute summaries of the risks associated with a change in a single variable in Z from a single level (quantile) to a second level (quantile), for the other variables in Z fixed to a specific level (quantile)

Usage

```
computeSingVarRisk(
  object,
  which.z,
  qs.diff = c(0.25, 0.75),
  q.fixed = c(0.25, 0.5, 0.75),
  nsamples = 1000,
  ...
)
```

Arguments

object	an object of class gpr obtained from **gvagpr** function.
which.z	vector indicating which variables (columns of Z) for which the summary should be computed
qs.diff	vector indicating the two quantiles q_1 and q_2 at which to compute f(z_{q2}) - f(z_{q1})
q.fixed	vector of quantiles at which to fix the remaining predictors in Z
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
...	unused argument

Value

a data frame containing the (posterior mean) estimate and posterior standard deviation of the single-predictor risk measures

References

Bobb J (2023). _bkmr: Bayesian Kernel Machine Regression_. R package version 0.2.2, <https://github.com/jenfb/bkmr>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
risks.singvar <- computeSingVarRisk(fout, which.z = 1:3, nsamples = 300)
risks.singvar$summary
plot(risks.singvar)
```

---

extractELBO *Extract ELBO from gpr, ggpr, or gpmim model fits*

Description

Compute the evidence lower bound (ELBO) at the fitted variational approximation for class "gpr" or "gpmim".

Usage

```
extractELBO(object, nsamples = 1000)
```

Arguments

`object`	an object of class `gpr` or `gpmim`.
`nsamples`	(positive integer), number of posterior samples to draw and save, defaults to 1000.

Details

The value is a Monte-Carlo estimate of the bound at the fitted q, averaged over `nsamples` fresh draws seeded from the fit, so the same object always gives the same number. It is not the bound the optimiser recorded along the way: that trace is `object$elbo_trace`, and its moving average, which the stopping rule reads, is `object$elbo_smooth`.

Which quantity the trace is differs by model. A Gaussian fit (`gvagpr`, `gvagpmim`, with or without random effects or mini-batches) records the expected log joint without the entropy of the variational distribution, so at the same parameters its trace sits above this value by minus the entropy; a Polya-Gamma fit records the full bound. The trace is also a moving average taken while the optimiser

was still climbing, so on a short run it lags well below the bound at the returned parameters. This function returns the full bound for every model.

The Monte-Carlo error depends on the model. On a Gaussian index fit the single-draw standard deviation of the bound is about 800 nats, which makes the default `nsamples` good to about 25 nats; compare fits at the same `nsamples` and raise it when the difference is small. The Polya-Gamma bounds are far less noisy.

For a multiple-index fit with a Polya-Gamma outcome (`gvaggpmim`) the value is the collapsed bound the fit climbed – the inducing values integrated out, the random effects, if any, as coordinates – evaluated the same way. Each draw solves the bound's inner fixed point for the Polya-Gamma parameter, so this is slower per draw than the other bounds.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

`extractPostSamps` *Extract Posterior Samples from gpr, ggpr, or gpmim model fits*

Description

Generate the posterior samples for class "gpr" or "gpmim"

Usage

```
extractPostSamps(object, nsamples = 1000)
```

Arguments

`object`	an object of class `gpr` or `gpmim`.
`nsamples`	(positive integer), number of posterior samples to draw and save, defaults to 1000.

Value

a data frame including posterior samples for all parameters. If `family` is `gaussian`, the posterior samples contain β, σ^2, λ_f, and γ (for an index model, ρ, ψ, γ and α in place of γ). For a `gvaggpr` fit the posterior samples also contain the inducing values $\bar{f}_j, j = 1, \ldots, \bar{n}$; a `gvaggpmim` fit has no such block, its bound integrating the inducing values out, and returns none. If `object$id` is not `NULL`, the data frame also includes $b_i, i = 1, \ldots, N$.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

extractRandomEffects *Extract the Random Effects of a Fitted Model*

Description

Returns the variational posterior means and standard deviations of the random intercepts (and slopes, when `random.slope` was given) of a model fitted by `gvagpr`, `gvaggpr`, `gvagpmim` or `gvaggpmim` with a cluster indicator `id`, one row per cluster, labelled by the levels of `id`.

Usage

```
extractRandomEffects(object)
```

Arguments

`object`	an object of class `gpr` or `gpmim` fitted with `id`.

Value

a list with `mean` and `sd`, each an `nID` by `q` matrix whose rows are the clusters and whose columns are the intercept and, when present, the slopes; and `Sigma.b`, the estimated covariance matrix of the random effects, evaluated at the variational mean.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

fitted.gpmim *Extract GPMIM Fitted Values*

Description

fitted is a generic function which extracts fitted values of nonparametric part from an object of class "gpmim"

Usage

```
## S3 method for class 'gpmim'
fitted(
  object,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

object	an object of class gpmim.
nsamples	(positive integer) number of draws from the variational posterior. Used only when variance = "total"; the fitted surface itself needs no draws. Default value is 1000.
cov	logical. If TRUE (the default) the covariance matrix of the nonparametric part is returned as well. This matrix is n by n, so for large samples it may dominate the memory used by the call; set cov = FALSE to return the fitted means only.
variance	one of "conditional" (the default) or "total", choosing which covariance fcov reports. See the Value section.
...	unused argument.

Value

fmean	the exposure–response surface at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$, where $\hat{\theta}$ is the componentwise median of the variational posterior. This is a plug-in and not the posterior mean $E[f \mid y]$, which would average $E[f \mid y, \theta]$ over $q(\theta)$. Returned in the order of the data supplied to the fitting function (also for random-effects fits, whose rows are stored sorted by cluster).
fcov	NULL if cov = FALSE. With variance = "conditional", the covariance of f at that same $\hat{\theta}$. It conditions on the parameters, so it drops $\mathrm{Var}_\theta(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$. It is usually the narrower of the two, but not necessarily at every point: the difference between them is not positive semi-definite. With variance = "total", the posterior covariance of f itself, obtained from the law of total variance over nsamples draws; that term has no closed form, so this option costs one $n \times n$ factorisation per draw and is much slower.

an object of class "gprfit", which has the associated method:

* plot (i.e., plot.gprfit)

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

---

fitted.gpr *Extract GPR and GGPR Model Fitted Values*

Description

fitted is a generic function which extracts fitted values of nonparametric part from an object of class "gpr"

Usage

```
## S3 method for class 'gpr'
fitted(
  object,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

object	an object of class gpr.
nsamples	(positive integer) number of draws from the variational posterior. Used only when variance = "total"; the fitted surface itself needs no draws. Default value is 1000.
cov	logical. If TRUE (the default) the covariance matrix of the nonparametric part is returned as well. This matrix is n by n, so for large samples it may dominate the memory used by the call; set cov = FALSE to return the fitted means only.
variance	one of "conditional" (the default) or "total", choosing which covariance fcov reports. See the Value section.
...	unused argument.

Value

fmean	the exposure–response surface at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$, where $\hat{\theta}$ is the componentwise median of the variational posterior. This is a plug-in and not the posterior mean $E[f \mid y]$, which would average $E[f \mid y, \theta]$ over $q(\theta)$. Returned in the order of the data supplied to the fitting function (also for random-effects fits, whose rows are stored sorted by cluster).
fcov	NULL if cov = FALSE. With variance = "conditional", the covariance of f at that same $\hat{\theta}$. It conditions on the parameters, so it drops $\mathrm{Var}_\theta(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$. It is usually the narrower of the two, but not necessarily at every point: the difference between them is not positive semi-definite. With variance = "total", the posterior covariance of f itself, obtained from the law of total variance over nsamples draws; that term has no closed form, so this option costs one $n \times n$ factorisation per draw and is much slower.

an object of class "gprfit", which has the associated method:

* plot (i.e., plot.gprfit)

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

`gvaggpmim` *Gaussian Variational Approximation to Generalized Gaussian Process Multiple Index Model*

Description

Fits the Bayesian kernel machine multiple index model for binomial and negative binomial outcomes by Gaussian variational approximation with Polya-Gamma augmentation, with optional cluster random intercepts and slopes.

Usage

```
gvaggpmim(
  y,
  X,
  Z,
  m.index = NULL,
  id = NULL,
  random.slope = NULL,
  family = binomial(),
  priors = list(),
  inits = list(),
  covstr = c("fullrank", "diagonal", "blockdiag", "sparseprec"),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

`y`	a vector of outcomes: 0/1 for `binomial`, non-negative integer counts for `negbinomial`.
`X`	a matrix of covariates that enter the model linearly.
`Z`	exposures: either a matrix, with `m.index` giving the grouping, or a list with one element per index.
`m.index`	a list of integer positions or exposure names, one element per index, used when `Z` is a matrix. Ignored, with a warning, when `Z` is a list.
`id`	optional vector of length `n` of cluster indicators for a model with random effects (a random intercept and, with `random.slope`, random slopes), as in `gvaggpr` and `gvagpmim`. `NULL` fits the fixed-effects model. The rows of a random-effects fit are stored sorted by cluster; `fitted` returns values in the order the data were given.
`random.slope`	index of the column(s) of `X` that carry a random slope. Read only together with `id`; without it a warning is given and the fixed-effects model is fitted.
`family`	`binomial()` or `negbinomial()`; the dispersion of the latter is estimated rather than fixed. A count outcome puts the inner fixed point for the Polya-Gamma parameter c under more strain than a binary one, because the linear predictor it works on is $\eta - \log r$; see `control$cit` and the section below.

priors — a list; lengthscale is 'horseshoe' (the default) or 'indep.normal', and the remaining elements give the hyperparameters, with their defaults in parentheses. alam (0.1) and blam (0.01) are the shape and rate of the gamma prior on the kernel amplitude λ_f. Under 'horseshoe', lam0 (1) is the scale of the half-Cauchy prior on each local scale λ_{ml}, tau0 (1) that of the per-index scale τ_m and taur0 (1) that of the global scale; under 'indep.normal', tau0 (10) is the standard deviation of the normal prior on each ρ_{ml}. For a count outcome s_theta (1) is the scale of the half-normal prior on $1/\sqrt{r}$, as in gvaggpr. With random effects vtaub (100) is the variance of the normal prior on the free elements of the Cholesky factor of Σ_b, as in gvaggpr. 'horseshoe' is the same prior gvagpmim uses, local scales and all: integrating the inducing values out frees the parameter budget to carry them.

inits — a list of starting values.

covstr — structure of the variational covariance. Without random effects: "fullrank" (the default) or "diagonal". Both recover the index equally well, but a diagonal covariance understates the posterior spread of the index weights by about a factor of two, and those weights are usually what is read; "diagonal" is the cheaper choice when only the fitted surface is wanted; "blockdiag" and "sparseprec" are treated as "fullrank". With random effects: "diagonal", "blockdiag" (one block per cluster and one for the shared parameters) or "sparseprec" (a sparse precision matrix that adds the dependence between each cluster and the shared parameters, updated by the natural gradient of Tan (2025)), as in gvaggpr; an unrestricted factor is not offered there, so "fullrank" becomes "blockdiag", silently when covstr was left at its default and with a warning when it was asked for.

control — a list of options for the optimiser, as in gvaggpr (max_iter, alpha0_mu, alpha0_L, beta, nws, nsp), including ninpts and knots for the inducing points and batch.size for the mini-batches. For a count outcome batch.size defaults to 100 and nbatch may be given instead, the two having to agree. For a binary outcome the split is not a setting: it is fixed at 150, and a batch.size or nbatch that is supplied is refused with a warning. cit (30) caps the inner fixed-point iteration for the Polya-Gamma parameter c inside the bound; the iteration exits as soon as it converges, which small binary samples do in about ten passes, while counts and larger binary samples run to the cap on most updates. The fraction that did is reported as c.unconverged; wherever it was measured, raising the cap did not move the answer.

minibatch — TRUE or FALSE (default TRUE): the likelihood is evaluated on a batch and the priors are scaled by the batch fraction, as in gvagpmim. The inducing-point kernel does not depend on the batch, so only the data sums are estimated from it; the cost per update stops depending on n. How large a batch has to be before this is free depends on the family: a binary outcome is held at 150 and refuses a smaller split, counts default to 100 and may be changed, and a binary outcome below five hundred observations is warned about rather than batched silently. See *Mini-batches and the sample size* below. The option does not apply to the random-effects models, which use the whole sample at every update.

verbose — print progress.

seed — random seed.

Details

The model is the multiple-index kernel machine of gvagpmim with a Polya-Gamma likelihood in place of the Gaussian one. Exposures enter through M indices, the mth being a weighted combi-

nation of the exposures in group m, and the kernel is $K_{ij} = \exp(-\sum_m \{(z_{im} - z_{jm})'\psi_m\}^2)$ with $\psi_m = D_m^{-1}\rho_m$. The last coordinate of each ρ_m is positive, which is what encodes $\sum_l \alpha_{ml} \geq 0$ and identifies the index.

Two length-scale priors are offered, the same two `gvagpmim` offers. `'horseshoe'` gives each ρ_{ml} a normal with a local scale $\lambda_{ml} \sim C^+(0, s)$, s being `priors$lam0`, on top of a per-index τ_m and a global τ_γ, so shrinkage acts on an exposure, on an index and overall; the last coordinate of each index is restricted to the positive half, which is what identifies the index. `'indep.normal'` adds no coordinates beyond ρ. Carrying the local scales is affordable here because the inducing values are integrated out of the bound rather than estimated – under the older formulation, which kept them, the local scales had to be integrated out instead to keep the optimisation stable.

Value

an object of class `gpmim` carrying `family`. A random-effects fit also carries `id`, `id.labels`, `nID`, `q`, `U`, `order` (the permutation that sorted the rows by cluster) and, with `random.slope`, `random.slope`; the cluster effects and their covariance are read with `extractRandomEffects`. Beyond the fields a `gvagpmim` fit has, it reports two diagnostics, neither of which raises a warning: `c.unconverged`, the fraction of updates on which the inner fixed point for c was not reached within `control$cit`, and `cond.K_IP`, the condition number of the inducing-point kernel at the fitted index weights. A large condition number is the normal state of an RBF kernel over a few dozen inducing points in a low-dimensional score space, not a symptom; the two ways the kernel actually loses information are caught by the kernel check at the end of the fit.

Mini-batches and the sample size

A Bernoulli observation carries much less information than a count, and a mini-batched binary index fit shrinks the surface when the batches are small or the sample is. The split for a binary outcome is therefore **fixed** at 150 observations per batch – a `batch.size` or `nbatch` passed in `control` is refused with a warning – and asking for mini-batches with fewer than five hundred binary observations warns and suggests `minibatch = FALSE`. A count outcome is not affected: its default batch size stays at 100 and no sample-size warning is given. The basic model `gvaggpr` does not shrink either. The difference is structural: the index model integrates the inducing values out of its bound and rebuilds the amplitude of the surface from the fitted hyperparameters, which inherit any bias a small batch puts in them, whereas the basic model carries the inducing values as a variational block.

Random effects

The random effects enter the linear predictor, $\eta = X\beta + Ub + f(z)$, with $b_i \sim N(0, \Sigma_b)$ per cluster and the Cholesky factor of Σ_b given a normal prior on its free elements, exactly as in `gvaggpr` and `gvagpmim`. They are variational coordinates, and the inducing values are still integrated out of the bound: b enters the bound only through the linear predictor, so the collapsed bound of the fixed-effects model carries over with $X\beta$ replaced by $X\beta + Ub$, plus the prior on b and its covariance. The latent surface is rebuilt from the fitted parameters with the cluster effects in the linear predictor, so `fitted` returns the population-level surface and not the surface plus the cluster effects. These fits use the whole sample at every update; mini-batches do not apply.

How the bound is built

The latent values at the inducing points are integrated out of the bound rather than carried as variational coordinates. That matters here in a way it does not for `gvaggpr`: an index model combines the exposures *before* the kernel, so `rho` moves the inducing points themselves and a value attached to one stops meaning the same thing. Kept as coordinates they and `rho` sit on a curved ridge that a

joint Gaussian cannot follow, and the fit settles on a degenerate solution in which the free coordinates of each index are driven to zero, only the exponentiated one survives, and the kernel collapses. Integrating them out removes the ridge; the latent process is then recovered in closed form when a surface is asked for, which is one approximation fewer than reading a variational $q(f_{IP})$ would be.

The starting direction for `rho` is taken from a quadratic logistic fit (an index enters its quadratic part as $\rho' zz' \rho$, so the leading eigenvector of the within-index block points along ρ). It costs a few thousandths of a second and only has to be roughly right: the scale does not matter and a poor direction is recovered from. Nothing here is user-facing.

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpr, computePPIPs

Examples

```
## Six exposures in three indices; only the first two carry signal, and
## within the first only As and Cd do.  This is the design the package's own
## tests use.
set.seed(1)
n  <- 150
g1 <- matrix(runif(n * 3, -1, 1), n, 3)
g2 <- matrix(runif(n * 2, -1, 1), n, 2)
g3 <- matrix(runif(n * 1, -1, 1), n, 1)
Z  <- cbind(As = g1[, 1], Cd = g1[, 2], Pb = g1[, 3],
            p1 = g2[, 1], p2 = g2[, 2], s1 = g3[, 1])
X  <- cbind(x1 = rnorm(n))
m.index <- list(1:3, 4:5, 6)

h   <- (2 * as.numeric(g1 %*% c(0.8, 0.6, 0)) + 0.5)^2 -
       (2 * as.numeric(g2 %*% c(0.7, 0.7)))^2
eta <- 0.3 * X[, 1] + h - mean(h)
y   <- rbinom(n, 1, plogis(eta))

## Below five hundred observations a binary index model is fitted on the
## whole sample.  The iteration caps in `ctl` keep this example short;
## drop them for a real analysis and let the stopping rule decide.
ctl <- list(ninpts = 20, max_iter = 2000, nws = 500, nsp = 30)

fout <- gvaggpmim(y, X, Z, m.index, family = binomial(), control = ctl,
                  minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fout)        # 'alpha.' rows are the index weights, 'gamma.' the scales
plot(fout)           # the smoothed bound; read the plateau, not the climb
computePPIPs(fout, nsamples = 300)   # needs a shrinkage lengthscale (the default)

## the latent surface at the observed exposures, and at new ones
fitted(fout, cov = FALSE)$fmean[1:5]
predict(fout, Z_new = Z[1:3, ], cov = FALSE)$fmean

## a count outcome
yc <- rnbinom(n, size = 2, mu = exp(1 + 0.3 * eta))
fc <- gvaggpmim(yc, X, Z, m.index, family = negbinomial(), control = ctl,
                minibatch = FALSE, verbose = FALSE, seed = 1)
```

```
summary(fc)          # 'theta' is the estimated dispersion

## a random intercept per cluster, under the sparse precision structure
id <- rep(1:15, each = 10)
yr <- rbinom(n, 1, plogis(eta + rnorm(15, sd = 0.6)[id]))
fr <- gvaggpmim(yr, X, Z, m.index, id = id, family = binomial(),
                covstr = "sparseprec", control = ctl, verbose = FALSE, seed = 1)
extractRandomEffects(fr)$Sigma.b

## Mini-batches are the default from five hundred observations; a binary
## index model splits at 150 and takes no batch.size (see Details).
```

---

gvaggpr *Gaussian Variational Approximation to Generalized Gaussian Process Regression*

Description

Fits the Bayesian kernel machine regression for generalized linear model using Gaussian variational approximation algorithm.

Usage

```
gvaggpr(
  y,
  X,
  Z,
  id = NULL,
  random.slope = NULL,
  group = NULL,
  family = binomial(),
  priors = list(),
  inits = list(),
  covstr = c("diagonal", "fullrank", "blockdiag", "sparseprec"),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

y	a vector of response of length n.
X	an n-by-p matrix of covariates for parametric term. Should not contain an intercept.
Z	an n-by-M matrix of predictor variables to be included in nonparametric part.
id	optional vector (of length n) of grouping factors for fitting a model with random effects (including both a random intercept and a random slope). If NULL then no random effects will be included.

`random.slope` a column index of the matrix (`X`) including covariates for random slope. If `NULL` and `id` is given, the model considers the random intercept only. Two caveats. (i) A slope the design does not need inflates its variance component under weak within-cluster information, a property of the Gaussian variational family; point estimates, selection, the surface and `var(b0)` are unaffected. (ii) Fixed-effect posterior intervals from a slope model are slightly narrow, because the coupling row of the sparse precision converges last; point estimates and the variance components are unaffected.

`group` optional grouping of the columns of `Z` for the grouped beta-prime prior: either a vector with one group label per column, or a list with one element per group holding column indices or names (columns that the list leaves out form groups of their own). Giving `group` selects `priors$lengthscale = 'group.betaprime'`; naming another prior together with `group` is an error. The grouping is available with and without random effects.

`family` a description of the error distribution and link function. Either `binomial()` with the logit link, or `negbinomial()` with the log link, in which case the linear predictor is the log mean, as in `MASS::glm.nb`. The dispersion is always estimated; a starting value may be given through `inits$theta`. `MASS::negative.binomial(theta = )` is also accepted, its `theta` taken as that starting value, with a warning saying so.

`priors` a list giving the prior information. The list includes the following parameters (with default values in parentheses): `alam` (`10`) and `blam` (`1`) giving the shape and rate of the gamma prior on λ_f, `tau0` giving the scale of the prior on the inverse length-scales (1 under `'betaprime'`, 10 under `'normal'` and `'indep.normal'`), `s_theta` (`1`) giving the scale of the half-normal prior placed on $1/\sqrt{r}$, the standard deviation of the multiplicative random effect in the Poisson–gamma representation of the negative binomial. The element `lengthscale` selects the prior on the inverse length-scale parameters and is one of `'normal'` (a single length-scale shared by all exposures), `'indep.normal'` (one length-scale per exposure, the default when `ncol(Z) > 1`), `'betaprime'` (one per exposure with a continuous shrinkage prior, which performs variable selection) or `'group.betaprime'` (the shrinkage prior on the total of each group of exposures given in `group`, the default when `group` is given; the Dirichlet concentration of the shares within a group is `a` (`1`)). `'shrinkage'` is accepted as a deprecated alias for `'betaprime'`, and `'group.horseshoe'` as an alias for `'group.betaprime'`.

`inits` a named `list` of starting values for the variational means, given by name rather than by position, so that the order in which the blocks are stored does not matter. Every element is given on the natural scale and is logged internally where the parameter is positive; a scalar is recycled over a block. The names are `beta` (the regression coefficients), `lengthscale` (the inverse length-scales γ), `lambda` (the scale of the Gaussian process), `taur` (the global shrinkage scale, for the beta-prime priors), and, for the negative binomial family, `theta` (the dispersion). Anything not named starts at zero on the working scale. The inducing values and the random-effect covariance are set internally and are not exposed.

`covstr` Either "diagonal" (the default), "fullrank", "blockdiag", or "sparseprec", indicating which covariance structure of variational distribution is used. The "diagonal" option uses a fully factorized Gaussian for the approximation, and is available for every model. For a model without random effects, the "fullrank" option uses a Gaussian with an unrestricted lower triangular Cholesky factor. For the mixed effect models the parameter vector is partitioned by cluster and an unrestricted

factor is not offered: the "blockdiag" option uses a Gaussian whose covariance matrix is block diagonal, with one block per cluster and one block for the parameters shared by all clusters, and the "sparseprec" option uses a Gaussian with a sparse precision matrix, which adds the dependence between each cluster and the shared parameters. An option that does not apply to the model at hand falls back to the closest one that does, with a warning: "fullrank" becomes "blockdiag" when random effects are present, and "blockdiag" and "sparseprec" become "fullrank" when they are not.

`control` a named `list` of parameters to control the algorithm's behavior. The list includes the following parameters (with default values in parentheses): `max_iter (100000)` giving the maximum number of iterations, `batch.size (100)` giving the size of each mini-batch when `minibatch = TRUE`, or equivalently `nbatch` giving their number (supplying both is an error unless they agree), `alpha0_mu (1e-3)` and `alpha0_L (1e-4)` giving the base step sizes for the mean and Cholesky parameters, respectively. For the random-effects models, which use the whole sample at each iteration, `alpha0_L` defaults instead to `3e-3` with `covstr = "sparseprec"` and to `1e-3` otherwise; a value given in `control` is used as given, `beta (0.9)` giving the exponential moving-average coefficient used to smooth the (normalized) natural-gradient direction, `nws (2500)` giving rolling window size for calculating the moving average of the lower bounds, `nsp (100)` giving the maximum patience parameter, The loop stops once the rolling mean of the lower bound has failed to improve on its best value for `nsp` consecutive updates, and the message printed on stopping – "The algorithm has converged" – reports that this counter was reached, not that the bound has settled. A small `nsp` can therefore stop on a run of unlucky updates and still report success. Read the plateau in `plot()` before trusting a fit that stopped early, and raise `nsp` if the bound is still climbing. `ninpts (50)` giving the number of inducing points for sparse Gaussian process prior. `knots (NULL)` giving the inducing points for sparse Gaussian process prior.

`minibatch` TRUE or FALSE (default `TRUE`): if `TRUE` the data are split into batches of `control$batch.size` and `max_iter` counts epochs rather than updates. A split that would give fewer than two batches is not a mini-batch, so the full data are used instead and a warning is given; with the default batch size of 100 that is every sample smaller than 200. The option does not apply to the random effects models.

`verbose` TRUE or FALSE: flag indicating whether to print intermediate diagnostic information during the model fitting.

`seed` The seed for random number generation. The default is generated from 1 to the maximum integer supported by **R** on the machine.

Details

Jo et al. (2026+) proposed the Bayesian semiparametric generalized linear model with sparse Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_i \sim Bern(p_i), \log\left(p_i/(1+p_i)\right) = x_i^\top \beta + f(z_i),$$

$$f = (f(z_1), \ldots, f(z_D))^\top \sim GP(K^{-1}\bar{f}, \sigma^2\lambda_f \tilde{K}),\ z_i = (z_{i1}, \ldots, z_{iM})^\top,$$

$$\bar{f} = (f(\bar{z}_1), \ldots, f(\bar{z}_{\bar{D}}))^\top \sim GP(0, K_{\bar{D}}),$$

where $\bar{D} << D$, $K = K_D K_{\bar{D}\bar{D}}^{-1}$, $\tilde{K} = K_D - K_{D\bar{D}} K_{\bar{D}\bar{D}}^{-1} K_{\bar{D}D}$, and the kernel denotes the RBF kernel given as

1) Equal lengthscale parameter:

$$K_D = \left(\exp\left(-\gamma \sum_{m=1}^{M} \|z_i - z_j\|^2 \right) \right)_{i,j=1}^{D}$$

2) Varying lengthscale parameters:

$$K_D = \left(\exp\left(-\sum_{m=1}^{M} \gamma_m \|z_i - z_j\|^2 \right) \right)_{i,j=1}^{D}$$

For the parameters, the following priors are used:

$$\pi(\beta) \propto 1,$$

$$\pi(\lambda_f) = Gamma(a_\lambda, b_\lambda),$$

1) Normal prior:

$$\pi(\gamma) = N_+(0, \tau_0^2)$$

2) Independent Normal priors:

$$\pi(\gamma_m) = N_+(0, \tau_0^2),\ m = 1, \ldots, M$$

3) Beta-prime (continuous shrinkage) prior:

$$\gamma_m \mid \tau_\gamma \sim \mathrm{BetaPrime}\left(\frac{1}{2}, \frac{1}{2};\ \text{scale} = \tau_\gamma^2\right), \qquad m = 1, \ldots, M,$$

$$\pi(\gamma_m \mid \tau_\gamma) \propto \gamma_m^{-1/2} \left(1 + \gamma_m/\tau_\gamma^2\right)^{-1}, \qquad \gamma_m > 0,$$

$$\tau_\gamma \sim \mathcal{C}^+(0, \tau_0),$$

4) Grouped beta-prime prior, with the exposures partitioned into groups $g = 1, \ldots, G$ through `group`:

$$\gamma_m = T_{g(m)}\, w_m,\ m = 1, \ldots, M,$$

$$T_g \mid \tau_\gamma \sim \mathrm{BetaPrime}\left(\frac{1}{2}, \frac{1}{2};\ \text{scale} = \tau_\gamma^2\right),\ g = 1, \ldots, G,\ \ \tau_\gamma \sim \mathcal{C}^+(0, \tau_0),$$

$$(w_m)_{m \in g} \sim Dirichlet(a, \ldots, a),\ g = 1, \ldots, G,$$

so that the shrinkage acts on the total inverse length-scale T_g of a group and the Dirichlet divides that total among the group's members: a small a puts the total on few of them, $a = 1$ is uniform on the shares, and a group of one member has the beta-prime prior above. This is the Polya-Gamma counterpart of the grouped horseshoe of `gvagpr`, the local scales being integrated out here as they are in the beta-prime prior.

Here τ_0 and a are positive constants specified by the user.

For more details, see Jo and Lee (2026+).

Value

an object of class "gpr", which has the associated methods:

* extractELBO

* fitted (i.e., fitted.gpr)

* summary (i.e., summary.gpr)

* predict (i.e., predict.gpr)

* plot (i.e., plot.gpr)

* extractRandomEffects, for a fit with random effects

For a fit with random effects the rows of `y`, `X`, `Z` and `U` in the object are sorted by cluster, as the optimiser requires; `order` is the permutation that was applied and `id.labels` the cluster labels in the order of the integer codes in `id`. `fitted` returns its values in the order the data were supplied.

Author(s)

Seongil Jo and Woojoo Lee

References

Jo, S., and Lee, W. (2025+), "Gaussian variational approximation to Bayesian kernel machine regression for Logistic models", *preprint*.

Jo, S., Ye, S., Hanh, G., and Lee, W. (2026), "Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior", *Statistics and Computing*.

Polson, N. G., Scott, J. G., and Windle, J. (2013), "Bayesian Inference for Logistic Models using Polya-Gamma Latent Variables", *Journal of the American Statistical Association*, 108, 1339-1349.

Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.

Tan, L. S. L. and Nott D. J. (2018), "Gaussian variational approximation with sparse precision matrices", *Statistics and Computing*, 28, 259-275.

Tan, L. S. L. (2025), "Analytic natural gradient updates for Cholesky factor in Gaussian variational approximation", *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 00, 1-27.

See Also

extractELBO, fitted.gpr, predict.gpr, plot.gpr, summary.gpr

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
## a binary outcome
yb <- rbinom(n, 1, plogis(0.5 * X[, 1] + h))
fb <- gvaggpr(yb, X, Z, family = binomial(), control = list(ninpts = 30),
              minibatch = FALSE, verbose = FALSE, seed = 1)
```

```
summary(fb)
plot(fb)
extractELBO(fb, nsamples = 200)

## a count outcome; 'theta' in the summary is the estimated dispersion
yc <- rnbinom(n, size = 2, mu = exp(1 + 0.3 * h))
fc <- gvaggpr(yc, X, Z, family = negbinomial(), control = list(ninpts = 30),
              minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fc)


## a random intercept and a random slope on the covariate
id <- rep(1:12, each = 10)
b0 <- rnorm(12, sd = 0.6)
yr <- rbinom(n, 1, plogis(0.5 * X[, 1] + h + b0[id]))
fr <- gvaggpr(yr, X, Z, id = id, random.slope = 1, family = binomial(),
              covstr = "sparseprec", control = list(ninpts = 30),
              verbose = FALSE, seed = 1)
extractRandomEffects(fr)$Sigma.b

## the grouped beta-prime prior: z1 and z2 as one group, z3 as a group of its own
fg <- gvaggpr(yb, X, Z, group = list(pair = c("z1", "z2")), family = binomial(),
              control = list(ninpts = 30), minibatch = FALSE, verbose = FALSE, seed = 1)
computePPIPs(fg)
```

---

gvagpmim *Gaussian Variational Approximation to Gaussian Process Multiple Index Model*

Description

Fits the Bayesian kernel machine multiple index model using Gaussian variational approximation algorithm.

Usage

```
gvagpmim(
  y,
  X,
  Z,
  m.index = NULL,
  id = NULL,
  random.slope = NULL,
  priors = list(),
  covstr = c("diagonal", "fullrank", "blockdiag", "sparseprec"),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

y	a vector of response of length n.
X	a matrix of covariates for parametric term. Should not contain an intercept.
Z	a matrix or a list of predictor variables to be included in nonparametric part.
m.index	a list of column indices grouping the predictor variables in nonparametric part. If Z is a matrix, m.index must be given.
id	optional vector (of length n) of grouping factors for fitting a model with random effects (including both a random intercept and a random slope). If NULL then no random effects will be included.
random.slope	a column index of the matrix (X) including covariates for random slope. If NULL and id is given, the model considers the random intercept only.
priors	a list giving the prior information. The list includes the following parameters (with default values in parentheses): asig (0.001) and bsig (0.001) giving the hyper parameters for σ^2, alam (0.1) and blam (0.01) giving the hyper parameters for λ_f, lam0 (1) and tau0 (1) giving the hyper parameters of the horseshoe prior. The element lengthscale selects the prior on the reparameterized index coefficients and is one of 'indep.normal' (independent normal priors, the default) or 'horseshoe' (a continuous shrinkage prior, which performs variable selection). There is no isotropic option here, since the index parameterization already carries one component per exposure. 'normal' is accepted as a deprecated alias for 'indep.normal'.
covstr	Either "diagonal" (the default), "fullrank", "blockdiag", or "sparseprec", indicating which covariance structure of variational distribution is used. The "diagonal" option uses a fully factorized Gaussian for the approximation, and is available for every model. For a model without random effects, the "fullrank" option uses a Gaussian with an unrestricted lower triangular Cholesky factor. For the mixed effect models the parameter vector is partitioned by cluster and an unrestricted factor is not offered: the "blockdiag" option uses a Gaussian whose covariance matrix is block diagonal, with one block per cluster and one block for the parameters shared by all clusters, and the "sparseprec" option uses a Gaussian with a sparse precision matrix, which adds the dependence between each cluster and the shared parameters. An option that does not apply to the model at hand falls back to the closest one that does, with a warning: "fullrank" becomes "blockdiag" when random effects are present, and "blockdiag" and "sparseprec" become "fullrank" when they are not.
control	a named list of parameters to control the algorithm's behavior. The list includes the following parameters (with default values in parentheses): max_iter (100000) giving the maximum number of iterations, rho (0.95) giving the decaying constant, eps (1e-6) giving the small positive constant added to ensure the denominator of the step size is positive and the initial step size is nonzero, nws (2500) giving rolling window size for calculating the moving average of the lower bounds, batch.size (100) giving the size of each mini-batch when minibatch = TRUE, or equivalently nbatch giving their number (supplying both is an error unless they agree), and nsp (100) giving the maximum patience parameter. The loop stops once the rolling mean of the lower bound has failed to improve on its best value for nsp consecutive updates, and the message printed on stopping – "The algorithm has converged" – reports that this counter was reached, not that the bound has settled. A small nsp can therefore stop on a run of unlucky updates and still report success. Read the plateau in plot() before trusting a fit that stopped early, and raise nsp if the bound is still climbing.

`minibatch`	TRUE or FALSE (default `TRUE`): if `TRUE` the data are split into batches of `control$batch.size` and `max_iter` counts epochs rather than updates. A split that would give fewer than two batches is not a mini-batch, so the full data are used instead and a warning is given; with the default batch size of 100 that is every sample smaller than 200. The option does not apply to the random effects models.
`verbose`	TRUE or FALSE: flag indicating whether to print intermediate diagnostic information during the model fitting.
`seed`	The seed for random number generation. The default is generated from 1 to the maximum integer supported by **R** on the machine.

Details

Jo et al. (2026) proposed the Bayesian semiparametric regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_i = x_i^\top \beta + f^M(z_{i1}^\top \alpha_1, \ldots, z_{iM}^\top \alpha_M) + \epsilon_i, \;\; \epsilon_i \overset{iid}{\sim} N(0, \sigma^2),$$
$$f_i^M = f^M(z_{i1}^\top \alpha_1, \ldots, z_{iM}^\top \alpha_M),$$
$$f^M = (f_1^M, \ldots, f_D^M)^\top \sim GP(0, \sigma^2 \lambda_f K_D),$$

where K_D denotes the RBF kernel given as

Varying lengthscale parameters:

$$K_D = \left(\exp\left(-\sum_{m=1}^{M} ((z_{im} - z_{jm})^\top D_m^{-1} \gamma_m)^2 \right) \right)_{i,j=1}^{D}, \;\; \gamma_m = (\gamma_{m1}, \ldots, \gamma_{mL_m})^\top,$$

where D_m is a non-singular matrix for constraints.

Jo et al. (2026) also proposed the Bayesian semiparametric mixed effects regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_{ir} = x_{ir}^\top \beta + f^M(z_{ir1}^\top \alpha_1, \ldots, z_{irM}^\top \alpha_M) + u_{ir}^\top b_i + \epsilon_{ir}, \;\; \epsilon_{ir} \overset{iid}{\sim} N(0, \sigma^2),$$
$$b_i = (b_{i1}, \ldots, b_{iq})^\top \overset{iid}{\sim} N(0, \Sigma_b).$$

For the parameters, the following priors are used:

$$\pi(\beta) \propto 1,$$
$$\pi(\sigma^{-2}) = Gamma(a_\sigma, b_\sigma),$$
$$\pi(\lambda_f) = Gamma(a_\lambda, b_\lambda),$$

1) Independent Normal priors:

$$\pi(\gamma_{ml}) = N(0, \tau_0^2), \; m = 1, \ldots, M, \; l = 1, \ldots, L_m - 1$$
$$\pi(\gamma_{mL_m}) = N_+(0, \tau_0^2), \; m = 1, \ldots, M$$

2) Horseshoe prior:

$$\pi(\gamma_{ml} \mid \lambda_{ml}, \tau_m, \tau_\gamma) = N(0, \lambda_{ml}^2 \tau_m^2 \tau_\gamma^2), \; m = 1, \ldots, M, \; l = 1, \ldots, L_m - 1$$
$$\pi(\gamma_{mL_m} \mid \lambda_{mL_m}, \tau_m, \tau_\gamma) = N_+(0, \lambda_{mL_m}^2 \tau_m^2 \tau_\gamma^2), \; m = 1, \ldots, M$$
$$\pi(\lambda_{ml}) = C_+(0, \lambda_0), \; m = 1, \ldots, M, \; l = 1, \ldots, L_m,$$
$$\pi(\tau_m) = C_+(0, \tau_0), \; m = 1, \ldots, M,$$
$$\pi(\tau_\gamma) = C_+(0, \tau_{\gamma,0}),$$

where $a_\sigma, b_\sigma, a_\lambda, b_\lambda, \lambda_0, \tau_0$ and $\tau_{\gamma,0}$ are positive constants specified by users.

For more details, see Jo et al. (2026).

Value

an object of class "gpmim", which has the associated methods:

* extractELBO

* fitted (i.e., fitted.gpmim)

* summary (i.e., summary.gpmim)

* predict (i.e., predict.gpmim)

* plot (i.e., plot.gpmim)

Author(s)

Seongil Jo and Woojoo Lee

References

Jo, S., Ye, S., Hanh, G., and Lee, W. (2026), "Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior", *Statistics and Computing*, 36:79.

Titsias, M. K. and Lazaro-Gredilla, M. (2014), "Doubly stochastic variational Bayes for non-conjugate inference", *Proceedings of the 31st ICML*.

Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.

Chen, H., Zheng, L., Kontai, R. A., and Raskutti, G. (2022), "Gaussian process parameter estimation using mini-batch stochastic gradient descent: convergence guarantees and empirical benefits", *Journal of Machine Learning Research*, 23, 1-59.

McGee, G., Wilson, A., Webster, T. F., Coull, B. A. (2021), "Bayesian multiple index models for environmental mixtures", *Biometrics*, 1-13.

See Also

extractELBO, fitted.gpmim, predict.gpmim, plot.gpmim, summary.gpmim

Examples

```
## five exposures in three indices; the first index carries the surface, the
## second a weaker one, the third nothing
set.seed(1)
n <- 80
Z <- matrix(runif(n * 5, -1, 1), n, 5,
            dimnames = list(NULL, c("As", "Cd", "p1", "p2", "s1")))
X <- cbind(x = rnorm(n))
m.index <- list(metals = 1:2, phenols = 3:4, single = 5)
s1 <- as.numeric(Z[, 1:2] %*% c(0.8, 0.6))
s2 <- as.numeric(Z[, 3:4] %*% c(0.7, 0.7))
h <- (2 * s1 + 0.5)^2 - s2^2
y <- 0.5 * X[, 1] + h - mean(h) + rnorm(n, sd = 0.5)

## the horseshoe prior on the index coordinates gives inclusion probabilities
fout <- gvagpmim(y, X, Z, m.index, priors = list(lengthscale = "horseshoe"),
                 minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fout, nsamples = 100)  # 'alpha.' rows: index weights; 'gamma.': length-scales
plot(fout)
```

```
extractELBO(fout, nsamples = 100)
computePPIPs(fout, nsamples = 100)
```

---

gvagpr *Gaussian Variational Approximation to Gaussian Process Regression*

Description

Fits the Bayesian kernel machine regression using Gaussian variational approximation algorithm.

Usage

```
gvagpr(
  y,
  X,
  Z,
  id = NULL,
  random.slope = NULL,
  group = NULL,
  priors = list(),
  covstr = c("diagonal", "fullrank", "blockdiag", "sparseprec"),
  inits = list(),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

y	a vector of response of length n.
X	an n-by-p matrix of covariates for parametric term. Should not contain an intercept.
Z	an n-by-M matrix of predictor variables to be included in nonparametric part.
id	optional vector (of length n) of grouping factors for fitting a model with random effects (including both a random intercept and a random slope). If NULL then no random effects will be included.
random.slope	a column index of the matrix (X) including covariates for random slope. If NULL and id is given, the model considers the random intercept only.
group	optional grouping of the columns of Z for the grouped horseshoe prior: either a vector with one group label per column, or a list with one element per group holding column indices or names (columns that the list leaves out form groups of their own). Giving group selects priors$lengthscale = 'group.horseshoe'; naming another prior together with group is an error. The grouping is available with and without random effects, but not with mini-batches when random effects are present, as for the other priors.

priors — a list giving the prior information. The list includes the following parameters (with default values in parentheses): asig (0.001) and bsig (0.001) giving the hyper parameters for σ^2, alam (0.1) and blam (0.01) giving the hyper parameters for λ_f, lam0 (1) and tau0 (1) giving the hyper parameters of the horseshoe priors, and a (1) giving the Dirichlet concentration of the shares within a group under the grouped horseshoe prior. The element lengthscale selects the prior on the inverse length-scale parameters and is one of 'normal' (a single length-scale shared by all exposures), 'indep.normal' (one length-scale per exposure, the default when ncol(Z) > 1), 'horseshoe' (one per exposure with a continuous shrinkage prior, which performs variable selection) or 'group.horseshoe' (the shrinkage prior on the total of each group of exposures given in group, the default when group is given).

covstr — Either "diagonal" (the default), "fullrank", "blockdiag", or "sparseprec", indicating which covariance structure of variational distribution is used. The "diagonal" option uses a fully factorized Gaussian for the approximation, and is available for every model. For a model without random effects, the "fullrank" option uses a Gaussian with an unrestricted lower triangular Cholesky factor. For the mixed effect models the parameter vector is partitioned by cluster and an unrestricted factor is not offered: the "blockdiag" option uses a Gaussian whose covariance matrix is block diagonal, with one block per cluster and one block for the parameters shared by all clusters, and the "sparseprec" option uses a Gaussian with a sparse precision matrix, which adds the dependence between each cluster and the shared parameters. An option that does not apply to the model at hand falls back to the closest one that does, with a warning: "fullrank" becomes "blockdiag" when random effects are present, and "blockdiag" and "sparseprec" become "fullrank" when they are not.

inits — a named list of starting values for the variational means, given by name rather than by position. Every element is on the natural scale and is logged internally where the parameter is positive; a scalar is recycled over a block. The names are beta (the regression coefficients), lengthscale (the inverse length-scales γ), lambda (the scale of the Gaussian process), sigsq.eps (the error variance), and, for the horseshoe priors, tau and taur (the local shrinkage scales, one per exposure or, under "group.horseshoe", one per group, and the global scale). Anything not named starts at zero on the working scale. The random-effect covariance is set internally and is not exposed.

control — a named list of parameters to control the algorithm's behavior. The list includes the following parameters (with default values in parentheses): max_iter (100000) giving the maximum number of iterations, rho (0.95) giving the decaying constant, eps (1e-6) giving the small positive constant added to ensure the denominator of the step size is positive and the initial step size is nonzero, nws (2500) giving rolling window size for calculating the moving average of the lower bounds, batch.size (100) giving the size of each mini-batch when minibatch = TRUE, or equivalently nbatch giving their number (supplying both is an error unless they agree), and nsp (100) giving the maximum patience parameter. The loop stops once the rolling mean of the lower bound has failed to improve on its best value for nsp consecutive updates, and the message printed on stopping – "The algorithm has converged" – reports that this counter was reached, not that the bound has settled. A small nsp can therefore stop on a run of unlucky updates and still report success. Read the plateau in plot() before trusting a fit that stopped early, and raise nsp if the bound is still climbing.

minibatch — TRUE or FALSE (default TRUE): if TRUE the data are split into batches of control$batch.size and max_iter counts epochs rather than updates. A split that would give fewer

	than two batches is not a mini-batch, so the full data are used instead and a warning is given; with the default batch size of 100 that is every sample smaller than 200. The option does not apply to the random effects models.
`verbose`	TRUE or FALSE: flag indicating whether to print intermediate diagnostic information during the model fitting.
`seed`	The seed for random number generation. The default is generated from 1 to the maximum integer supported by **R** on the machine.

Details

Jo et al. (2026) proposed the Bayesian semiparametric regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_i = x_i^\top \beta + f(z_i) + \epsilon_i, \;\; \epsilon_i \overset{iid}{\sim} N(0, \sigma^2),$$

$$f = (f(z_1), \ldots, f(z_D))^\top \sim GP(0, \sigma^2 \lambda_f K_D), \; z_i = (z_{i1}, \ldots, z_{iM})^\top,$$

where K_D denotes the RBF kernel given as

1) Equal lengthscale parameter:

$$K_D = \left(\exp\left(-\gamma \sum_{m=1}^{M} \|z_i - z_j\|^2 \right) \right)_{i,j=1}^{D}$$

2) Varying lengthscale parameters:

$$K_D = \left(\exp\left(-\sum_{m=1}^{M} \gamma_m \|z_{im} - z_{jm}\|^2 \right) \right)_{i,j=1}^{D}$$

Jo et al. (2026) also proposed the Bayesian semiparametric mixed effects regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_{ir} = x_{ir}^\top \beta + f(z_{ir}) + u_{ir}^\top b_i + \epsilon_{ir}, \;\; \epsilon_{ir} \overset{iid}{\sim} N(0, \sigma^2),$$

$$b_i = (b_{i1}, \ldots, b_{iq})^\top \overset{iid}{\sim} N(0, \Sigma_b).$$

For the parameters, the following priors are used:

$$\pi(\beta) \propto 1,$$

$$\pi(\sigma^{-2}) = Gamma(a_\sigma, b_\sigma),$$

$$\pi(\lambda_f) = Gamma(a_\lambda, b_\lambda),$$

1) Normal prior:

$$\pi(\gamma) = N_+(0, \tau_0^2)$$

2) Independent Normal priors:

$$\pi(\gamma_m) = N_+(0, \tau_0^2), \; m = 1, \ldots, M$$

3) Horseshoe prior:

$$\pi(\gamma_m \mid \lambda_m, \tau_\gamma) = N_+(0, \lambda_m^2 \tau_\gamma^2), \; m = 1, \ldots, M$$

$$\pi(\lambda_m) = C_+(0, \lambda_0), \; m = 1, \ldots, M$$

$$\pi(\tau_\gamma) = C_+(0, \tau_0),$$

4) Grouped horseshoe prior, with the exposures partitioned into groups $g = 1, \ldots, G$ through `group`:

$$\gamma_m = T_{g(m)}\, w_m,\ m = 1, \ldots, M,$$

$$\pi(T_g \mid \lambda_g, \tau_\gamma) = N_+(0, \lambda_g^2 \tau_\gamma^2),\ \pi(\lambda_g) = C_+(0, \lambda_0),\ g = 1, \ldots, G,\ \ \pi(\tau_\gamma) = C_+(0, \tau_0),$$

$$(w_m)_{m \in g} \sim Dirichlet(a, \ldots, a),\ g = 1, \ldots, G,$$

so that the horseshoe acts on the total inverse length-scale T_g of a group and the Dirichlet divides that total among the group's members: a small a puts the total on few of them, $a = 1$ is uniform on the shares, and a group of one member has the horseshoe prior above.

Here $a_\sigma, b_\sigma, a_\lambda, b_\lambda, \lambda_0, \tau_0$ and a are positive constants specified by users.

For more details, see Jo et al. (2026).

Value

an object of class "gpr", which has the associated methods:

* extractELBO

* fitted (i.e., fitted.gpr)

* summary (i.e., summary.gpr)

* predict (i.e., predict.gpr)

* plot (i.e., plot.gpr)

* extractRandomEffects, for a fit with random effects

For a fit with random effects the rows of `y`, `X`, `Z` and `U` in the object are sorted by cluster, as the optimiser requires; `order` is the permutation that was applied and `id.labels` the cluster labels in the order of the integer codes in `id`. `fitted` returns its values in the order the data were supplied.

Author(s)

Seongil Jo and Woojoo Lee

References

Jo, S., Ye, S., Hanh, G., and Lee, W. (2026), "Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior", *Statistics and Computing*, 36:79.

Titsias, M. K. and Lazaro-Gredilla, M. (2014), "Doubly stochastic variational Bayes for non-conjugate inference", *Proceedings of the 31st ICML.*

Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.

Tan, L. S. L. and Nott D. J. (2018), "Gaussian variational approximation with sparse precision matrices", *Statistics and Computing*, 28, 259-275.

Chen, H., Zheng, L., Kontai, R. A., and Raskutti, G. (2022), "Gaussian process parameter estimation using mini-batch stochastic gradient descent: convergence guarantees and empirical benefits", *Journal of Machine Learning Research*, 23, 1-59.

Tan, L. S. L. and Nott, D. J. (2018), "Gaussian variatioal approximation with sparse precision matrices", *Statistics and Computing*, 28, 259-275.

See Also

extractELBO, fitted.gpr, predict.gpr, plot.gpr, summary.gpr

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fout)
plot(fout)                    # the smoothed bound along the optimisation
extractELBO(fout, nsamples = 200)
fitted(fout, cov = FALSE)$fmean[1:5]
predict(fout, Z_new = Z[1:3, ], cov = FALSE)$fmean


## a random intercept per cluster
id <- rep(1:12, each = 10)
yr <- y + rnorm(12, sd = 0.5)[id]
fr <- gvagpr(yr, X, Z, id = id, covstr = "blockdiag", verbose = FALSE, seed = 1)
extractRandomEffects(fr)$Sigma.b

## the grouped horseshoe: z1 and z2 as one group, z3 as a group of its own
fg <- gvagpr(y, X, Z, group = list(pair = c("z1", "z2")), minibatch = FALSE,
             verbose = FALSE, seed = 1)
computePPIPs(fg)
```

`metals` *Urinary metal concentrations and hypertension in NHANES*

Description

Adults aged 20 and over from four cycles of the National Health and Nutrition Examination Survey (2011–2012 through 2017–2018) who were part of the urinary metals subsample and had complete records on the variables below. Concentrations are as reported by the survey and are not transformed; the analysis in the package vignette and in Jo, Ye, Hahn and Lee (2026) takes logarithms and standardizes them.

Usage

```
metals
```

Format

A data frame with 5809 rows and 23 columns.

Ba, Cd, Co, Cs, Mo, Mn, Pb, Sb, Sn, Tl, W urinary concentrations of barium, cadmium, cobalt, caesium, molybdenum, manganese, lead, antimony, tin, thallium and tungsten, in ug/L.

sbp, dbp systolic and diastolic blood pressure in mmHg, averaged over the available readings.

meds logical: reported use of antihypertensive medication.

htn hypertension, defined as `sbp >= 140`, `dbp >= 90` or `meds`. Redefine it from `sbp`, `dbp` and `meds` to use another threshold.

age age in years.

sex factor, `"male"` or `"female"`.

race factor with levels `"MexAm"`, `"OthHisp"`, `"NHWhite"`, `"NHBlack"`, `"NHAsian"` and `"Other"`.

pir ratio of family income to the poverty threshold.

bmi body mass index in kg/m^2.

ucr urinary creatinine in mg/dL.

cotinine serum cotinine in ng/mL.

cycle factor giving the survey cycle.

Source

National Health and Nutrition Examination Survey, cycles 2011–2012, 2013–2014, 2015–2016 and 2017–2018. https://wwwn.cdc.gov/nchs/nhanes/

References

Jo S, Ye S, Hahn G, Lee W (2026). Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior. *Statistics and Computing*, **36**, 79.

Examples

```
data(metals)
table(metals$htn)
summary(metals$Pb)
```

---

negbinomial *Negative binomial family for count outcomes*

Description

A family object for the negative binomial distribution with the log link, for use with `gvaggpr` and `gvaggpmim`. The linear predictor is the log mean, as in `stats::glm` and `MASS::glm.nb`, so that `exp(beta)` is a rate ratio.

Usage

```
negbinomial(link = "log")
```

Arguments

`link`	the link function. Only `"log"` is available.

Details

The dispersion is always estimated: overdispersion is the reason to use a negative binomial model in the first place, so there is no option to hold it fixed, and the family therefore takes no dispersion argument. A starting value may be supplied through `inits$theta` in the call to `gvaggpr` or `gvaggpmim`; see the examples. The scale of the half-normal prior on $1/\sqrt{r}$ is `priors$s_theta` in both.

`MASS::negative.binomial(theta = )` is also accepted, with its `theta` taken as a starting value rather than as a fixed dispersion, and a warning to that effect.

Value

an object of class `"family"`.

See Also

`gvaggpr`, `gvaggpmim`

Examples

```
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3)
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- rnbinom(n, size = 2, mu = exp(1 + 0.3 * h))

fit <- gvaggpr(y, X, Z, family = negbinomial(), control = list(ninpts = 30),
               minibatch = FALSE, verbose = FALSE, seed = 1)
fit$theta.hat            # the estimated dispersion (true value 2)


## with a starting value for the dispersion
fit2 <- gvaggpr(y, X, Z, family = negbinomial(), inits = list(theta = 5),
                control = list(ninpts = 30), minibatch = FALSE, verbose = FALSE,
                seed = 1)

## a count outcome in the multiple index model (the iteration caps keep the
## example short; drop them for a real analysis)
m.index <- list(1:2, 3)
fitm <- gvaggpmim(y, X, Z, m.index, family = negbinomial(),
                  control = list(ninpts = 20, max_iter = 2000, nws = 500, nsp = 30),
                  minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fitm)
```

`plot.gpmim` *Plot Diagnostics for a gpmim Object*

Description

Provides a plot of the smoothed evidence lower bound (ELBO) against iterations for checking the convergence.

Usage

```
## S3 method for class 'gpmim'
plot(x, from = 1, ...)
```

Arguments

x	gpmim object, result of **gvagpmim**.
from	(positive integer) first iteration to show. The default, 1, plots the whole trace.
...	unused argument.

Details

The bound rises steeply over the first few iterations, often by several orders of magnitude, and on the full range that early climb takes up the whole vertical axis: the plateau where convergence is actually decided is then flat to within a pixel. Use from to start the plot after the climb and read the plateau.

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

---

plot.gpr *Plot Diagnostics for a gpr Object*

Description

Provides a plot of the smoothed evidence lower bound (ELBO) against iterations for checking the convergence.

Usage

```
## S3 method for class 'gpr'
plot(x, from = 1, ...)
```

Arguments

x	gpr object, result of **gvagpr** or **gvaggpr**.
from	(positive integer) first iteration to show. The default, 1, plots the whole trace.
...	unused argument.

Details

The bound rises steeply over the first few iterations, often by several orders of magnitude, and on the full range that early climb takes up the whole vertical axis: the plateau where convergence is actually decided is then flat to within a pixel. Use from to start the plot after the climb and read the plateau.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

---

plot.gprfitBivar *Plot bivariate predictor-response function on a new grid of points*

Description

Provides a plot of bivariate predictor-response function on a new grid of points

Usage

```
## S3 method for class 'gprfitBivar'
plot(x, ...)
```

Arguments

x	gpr object, result of **predictorResponseBivar**.
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr predictorResponseBivar

---

plot.gprfitBivarLevels *Plot interactions*

Description

Provides a plot of the predictor-response function of a single predictor in Z for the second predictor in Z fixed at various quantiles

Usage

```
## S3 method for class 'gprfitBivarLevels'
plot(x, ...)
```

Arguments

x	gpr object, result of **predictorResponseBivarLevels**.
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr predictorResponseBivar

`plot.gprfitUnivar` *Plot univariate predictor-response function on a new grid of points*

Description

Provides a plot of univariate predictor-response function on a new grid of points

Usage

```
## S3 method for class 'gprfitUnivar'
plot(x, ...)
```

Arguments

`x`	gpr object, result of **predictorResponseUnivar**.
`...`	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr predictorResponseUnivar

`plot.risks` *Plot Risk Summaries*

Description

Provides a plot of risk summaries

Usage

```
## S3 method for class 'risks'
plot(x, ...)
```

Arguments

`x`	`risks` object, result of `computeOverallRisk`, `computeSingVarRisk`, or `computeSingVarInt`
`...`	unused argument

Author(s)

Seongil Jo

See Also

gvagpr computeOverallRisk computeSingVarRisk computeSingVarInt

---

`predict.gpmim` *Extract GPMIM Predicted Values*

Description

predicted is a generic function which extracts predicted values for nonparametric part from an object of class "gpmim"

Usage

```
## S3 method for class 'gpmim'
predict(
  object,
  Z_new,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

`object`	an object of class `gpmim`.
`Z_new`	a matrix of new predictor values at which to predict new `f`, where each row represents a new observation.
`nsamples`	(positive integer) number of draws from the variational posterior. Used only when `variance = "total"`; the predictive surface itself needs no draws. Default value is 1000.
`cov`	logical; if `TRUE` (the default) the covariance of the new `f` is returned as well. With many new points this is the dominant cost, so set it to `FALSE` when only the predictive mean is wanted.
`variance`	one of `"conditional"` (the default) or `"total"`, choosing which covariance `fcov` reports. See the Value section.
`...`	additional arguments affecting the predictions produced.

Value

`fmean`	the exposure–response surface at the new points, evaluated at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$. This is a plug-in and not the posterior mean $E[f \mid y]$.

`fcov` `NULL` if `cov = FALSE`. With `variance = "conditional"`, the covariance of the new f at that same $\hat{\theta}$. It drops $\mathrm{Var}_\theta(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$, so it is usually the narrower of the two but not necessarily at every point. With `variance = "total"`, the posterior covariance itself, by the law of total variance over `nsamples` draws; that costs one factorisation per draw and is much slower.

an object of class "gprfit", which has the associated method:

* plot (i.e., plot.gprfit)

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

`predict.gpr` *Extract GPR and GGPR Model Predicted Values*

Description

predicted is a generic function which extracts predicted values for nonparametric part from an object of class "gpr"

Usage

```
## S3 method for class 'gpr'
predict(
  object,
  Z_new,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

`object`	an object of class `gpr`.
`Z_new`	a matrix of new predictor values at which to predict new `f`, where each row represents a new observation.
`nsamples`	(positive integer) number of draws from the variational posterior. Used only when `variance = "total"`; the predictive surface itself needs no draws. Default value is 1000.
`cov`	logical; if `TRUE` (the default) the covariance of the new `f` is returned as well. With many new points this is the dominant cost, so set it to `FALSE` when only the predictive mean is wanted.
`variance`	one of `"conditional"` (the default) or `"total"`, choosing which covariance `fcov` reports. See the Value section.
`...`	additional arguments affecting the predictions produced.

Value

`fmean` the exposure–response surface at the new points, evaluated at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$. This is a plug-in and not the posterior mean $E[f \mid y]$.

`fcov` `NULL` if `cov = FALSE`. With `variance = "conditional"`, the covariance of the new f at that same $\hat{\theta}$. It drops $\mathrm{Var}_\theta(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$, so it is usually the narrower of the two but not necessarily at every point. With `variance = "total"`, the posterior covariance itself, by the law of total variance over `nsamples` draws; that costs one factorisation per draw and is much slower.

an object of class "gprfit", which has the associated method:

* plot (i.e., plot.gprfit)

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

`predictorResponseBivar` *Predict the exposure-response function at a new grid of points*

Description

Predict the exposure-response function at a new grid of points

Usage

```
predictorResponseBivar(
  fit,
  z.pairs = NULL,
  ngrid = 50,
  q.fixed = 0.5,
  nsamples = 1000,
  min.plot.dist = 0.5,
  center = TRUE,
  verbose = TRUE,
  ...
)
```

Arguments

`fit`	an object of class `gpr`
`z.pairs`	data frame showing which pairs of predictors to plot
`ngrid`	number of grid points in each dimension
`q.fixed`	a second quantile at which to compare the estimated `f` function

nsamples	(positive integer) number of posterior samples. Default value is 1000
min.plot.dist	specifies a minimum distance that a new grid point needs to be from an observed data point in order to compute the prediction; points further than this will not be computed
center	flag for whether to scale the exposure-response function to have mean zero
verbose	TRUE or FALSE: flag of whether to print intermediate output to the screen
...	additional arguments affecting the predictions produced

Value

a long data frame with the name of the first predictor, the name of the second predictor, the value of the first predictor, the value of the second predictor, the posterior mean estimate, and the posterior standard deviation of the estimated exposure response function

References

Bobb J (2023). _bkmr: Bayesian Kernel Machine Regression_. R package version 0.2.2, <https://github.com/jenfb/bkmr>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
## every pair of exposures on a 10-by-10 grid
pred.resp.bivar <- predictorResponseBivar(fit = fout, min.plot.dist = 1, ngrid = 10,
                                          nsamples = 300, verbose = FALSE)
head(pred.resp.bivar)
```

---

predictorResponseBivarLevels

Plot cross-sections of the bivariate predictor-response function

---

Description

Function to plot the f function of a particular variable at different levels (quantiles) of a second variable

Usage

```
predictorResponseBivarLevels(
  object,
  Z = NULL,
  qs = c(0.25, 0.5, 0.75),
  both_pairs = TRUE
)
```

Arguments

object	object obtained from running the function predictorResponseBivar
Z	an n-by-M matrix of predictor variables to be included in nonparametric part.
qs	vector of quantiles at which to fix the second variable
both_pairs	flag indicating whether, if h(z1) is being plotted for z2 fixed at different levels, that they should be plotted in the reverse order as well (for h(z2) at different levels of z1)

Value

a long data frame with the name of the first predictor, the name of the second predictor, the value of the first predictor, the quantile at which the second predictor is fixed, the posterior mean estimate, and the posterior standard deviation of the estimated exposure response function

References

Bobb J (2023). _bkmr: Bayesian Kernel Machine Regression_. R package version 0.2.2, <https://github.com/jenfb/bkmr>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
## every pair of exposures on a 10-by-10 grid
pred.resp.bivar <- predictorResponseBivar(fit = fout, min.plot.dist = 1, ngrid = 10,
                                          nsamples = 300, verbose = FALSE)
head(pred.resp.bivar)
## the surfaces sliced at quantiles of the second exposure of each pair
pred.resp.bivar.levels <- predictorResponseBivarLevels(pred.resp.bivar, Z = Z,
                                                       qs = c(0.1, 0.5, 0.9))
plot(pred.resp.bivar.levels)
```

---

predictorResponseBivarPair

Predict bivariate predictor-response function on a new grid of points

---

Description

Predict bivariate predictor-response function on a new grid of points

Usage

```
predictorResponseBivarPair(
  fit,
  whichz1 = 1,
  whichz2 = 2,
  whichz3 = NULL,
  prob = 0.5,
  q.fixed = 0.5,
  nsamples = 1000,
  ngrid = 50,
  min.plot.dist = 0.5,
  center = TRUE,
  ...
)
```

Arguments

fit	an object of class gpr
whichz1	vector identifying the first predictor that (column of Z) should be plotted
whichz2	vector identifying the second predictor that (column of Z) should be plotted
whichz3	vector identifying the third predictor that will be set to a pre-specified fixed quantile (determined by prob)
prob	pre-specified quantile to set the third predictor (determined by whichz3); defaults to 0.5 (50th percentile)
q.fixed	a second quantile at which to compare the estimated f function
nsamples	(positive integer) number of posterior samples. Default value is 1000
ngrid	number of grid points in each dimension
min.plot.dist	specifies a minimum distance that a new grid point needs to be from an observed data point in order to compute the prediction; points further than this will not be computed
center	flag for whether to scale the exposure-response function to have mean zero
...	additional arguments affecting the predictions produced

Value

a data frame with value of the first predictor, the value of the second predictor, the posterior mean estimate, and the posterior standard deviation

References

Bobb J (2023). _bkmr: Bayesian Kernel Machine Regression_. R package version 0.2.2, <https://github.com/jenfb/bkmr>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
```

```
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
## a 10-by-10 grid keeps the example quick
pred.resp.bivar12 <- predictorResponseBivarPair(fit = fout, min.plot.dist = 1,
                                                ngrid = 10, nsamples = 300)
head(pred.resp.bivar12)
```

`predictorResponseUnivar` *Predict univariate predictor-response function on a new grid of points*

Description

Predict univariate predictor-response function on a new grid of points

Usage

```
predictorResponseUnivar(
  fit,
  which.z = 1:ncol(Z),
  ngrid = 50,
  q.fixed = 0.5,
  nsamples = 1000,
  min.plot.dist = Inf,
  center = TRUE,
  z.names = colnames(Z),
  ...
)
```

Arguments

`fit`	an object of class gpr
`which.z`	vector identifying which predictors (columns of Z) should be plotted
`ngrid`	number of grid points to cover the range of each predictor (column in Z)
`q.fixed`	a second quantile at which to compare the estimated f function
`nsamples`	(positive integer) number of posterior samples. Default value is 1000
`min.plot.dist`	specifies a minimum distance that a new grid point needs to be from an observed data point in order to compute the prediction; points further than this will not be computed
`center`	flag for whether to scale the exposure-response function to have mean zero
`z.names`	a vector of names of predictors Z. Default values are colnames(Z).
`...`	additional arguments affecting the predictions produced.

Value

a long data frame with the predictor name, predictor value, posterior mean estimate, and posterior standard deviation

References

Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.

Bobb J (2023). _bkmr: Bayesian Kernel Machine Regression_. R package version 0.2.2, <https://github.com/jenfb/bkmr>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
pred.resp.univar <- predictorResponseUnivar(fout, ngrid = 20, nsamples = 300)
plot(pred.resp.univar)
```

---

print.gpmim *Print basic summary of gpmim and ggpmim model fit*

Description

print method for class "gpmim"

Usage

```
## S3 method for class 'gpmim'
print(x, ...)
```

Arguments

x	an object of class gpmim.
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

print.gpr *Print basic summary of gpr and ggpr model fit*

Description

print method for class "gpr"

Usage

```
## S3 method for class 'gpr'
print(x, ...)
```

Arguments

x	an object of class gpr.
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

summary.gpmim *Summarizing gpmim and ggpmim model fits*

Description

summary method for class "gpmim"

Usage

```
## S3 method for class 'gpmim'
summary(
  object,
  q = c(0.025, 0.975),
  digits = 5,
  nsamples = 1000,
  ntuning = 10,
  ...
)
```

Arguments

object	an object of class gpmim
q	quantiles of posterior distribution (credible interval) to show
digits	the number of digits to show when printing
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
ntuning	the number of chosen values of the tuning parameter for variable selection, defaults to 10
...	unused argument.

Value

the table of posterior summaries of the model parameters – mean, standard deviation and the quantiles in q – returned invisibly; summary is called for what it prints. The rows are the regression coefficients, the error variance, the kernel amplitude lambda, and then three blocks that describe the index structure. gamma.<index> is the length-scale of an index, $\gamma_m = \sum_l \psi_{ml}^2$, the multiple-index counterpart of the per-exposure length-scales that summary.gpr reports. psi.<exposure> is the weight of one exposure inside its index on the same scale, signed, so that an interval covering zero says the exposure does not enter the index. alpha.<exposure> is that weight normalised, $\alpha_m = \psi_m / \|\psi_m\|$, which gives the composition of an index but not its size: it lies on the unit sphere, so a dead index still shows weights of order one. Pseudo posterior inclusion probabilities are printed for the shrinkage priors and are obtained on their own with computePPIPs.

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

---

summary.gpr *Summarizing gpr and ggpr model fits*

Description

summary method for class "gpr"

Usage

```
## S3 method for class 'gpr'
summary(
  object,
  q = c(0.025, 0.975),
  digits = 5,
  nsamples = 1000,
  ntuning = 10,
  ...
)
```

Arguments

object	an object of class gpr
q	quantiles of posterior distribution (credible interval) to show
digits	the number of digits to show when printing
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
ntuning	the number of chosen values of the tuning parameter for variable selection, defaults to 10
...	unused argument.

Value

the table of posterior summaries of the model parameters – mean, standard deviation and the quantiles in q – returned invisibly; summary is called for what it prints. The same table is returned whichever length-scale prior was fitted, so that code downstream does not have to branch on it. Pseudo posterior inclusion probabilities are printed for the shrinkage priors and are obtained on their own with computePPIPs.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

Index

∗ **datasets**
metals, 29

computeOverallRisk, 2
computePPIPs, 3
computeSingVarInt, 5
computeSingVarRisk, 6

extractELBO, 7
extractPostSamps, 8
extractRandomEffects, 9

fitted.gpmim, 9
fitted.gpr, 10

gvaggpmim, 12, *30, 31*
gvaggpr, *14*, 16, *30, 31*
gvagpmim, 21
gvagpr, 25

metals, 29

negbinomial, 30

plot.gpmim, 31
plot.gpr, 32
plot.gprfitBivar, 33
plot.gprfitBivarLevels, 33
plot.gprfitUnivar, 34
plot.risks, 34
predict.gpmim, 35
predict.gpr, 36
predictorResponseBivar, 37, *39*
predictorResponseBivarLevels, 38
predictorResponseBivarPair, 39
predictorResponseUnivar, 41
print.gpmim, 42
print.gpr, 43

summary.gpmim, 43
summary.gpr, 44